JAVA OOP LABORATORY EXERCISE

Campus Café Ordering System

Student Names: Aira Jefferson T. Cabano and Jeffrey Lim
Section: E
Date: November 10, 2026

1. Introduction

Our project is a simple Campus Café Ordering System made using Java. It allows a customer to order drinks, pastries, and meals. Each item has its own price, and student customers can get a 10% discount. Customers can also earn loyalty points from their orders.

2. OOP Concepts Used

Classes and Objects: We used seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. These classes help organize the parts of our program.

Abstraction: MenuItem is an abstract class. It has a computePrice() method that is completed by Beverage, Pastry, and Meal.

Encapsulation: We used private and protected variables. We also used getters and setters to access and change information safely.

Inheritance: Beverage, Pastry, and Meal inherit common information from the MenuItem class. This helps us avoid writing the same code again.

Polymorphism: The three menu item classes have their own version of computePrice() and getDescription(). This allows each item to work in its own way.

Method Overloading: We used methods with the same name but different parameters. Examples are addPoints() in Customer and computeTotal() in Order.

Constructors: We used default and parameterized constructors to give objects their starting values. Customer also has three constructors.

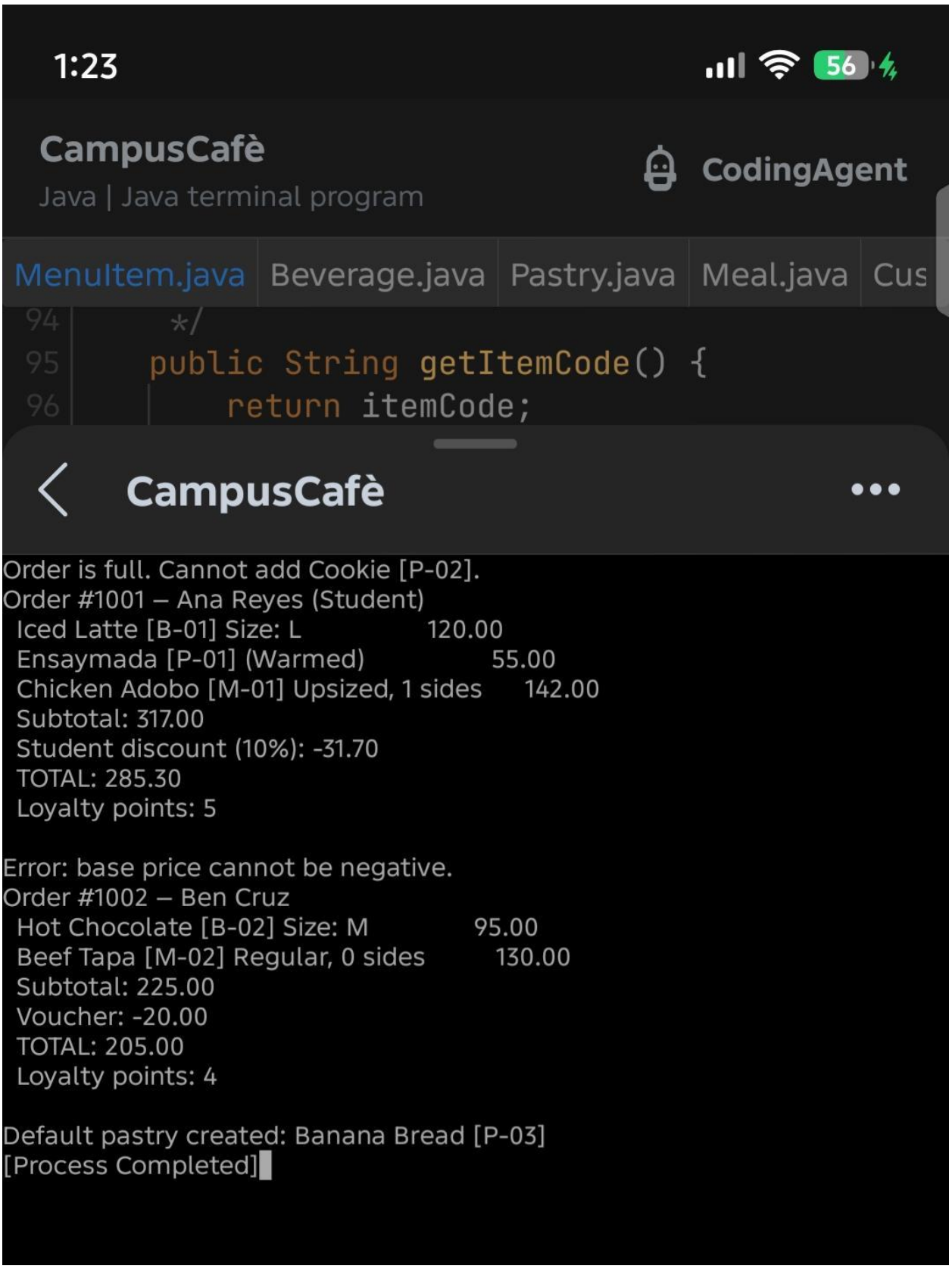
Conditional Statements: We used if and if-else statements to check prices, choose drink sizes, add extra fees, apply discounts, and limit orders to three items.

Object Interaction: The Order class connects the Customer and the menu items. It gets the prices, prints the receipt, and adds loyalty points to the customer.

Naming Conventions: We used capital letters at the beginning of class names and camelCase for variables and methods. We also used proper indentation to make the code easier to read.

3. Program Output

The program shows two sample orders. The first order is for a student and shows the 10% discount. It also tries to add a fourth item, but the program does not allow it because an order can only have three items. The second order shows how a voucher is subtracted from the total. The program also shows an error message when a negative price is entered.



4. Challenges and Solutions

One challenge we encountered was limiting the order to only three items without using an ArrayList. We solved this by using three separate item variables in the Order class. Another

challenge was calculating different prices for drinks, pastries, and meals. We solved this by using method overriding in each class. We also added conditions to prevent invalid values such as negative prices.

5. Conclusion

Through this activity, we learned how to use the basic concepts of Java Object-Oriented Programming. We learned how classes and objects work together, how inheritance can reuse code, and how methods can have different behaviors. This activity also helped us practice writing and organizing a Java program.